

Chapitre 3

Synthèse du Chapitre 3 : Les structures de contrôle en PL/SQL

Ce chapitre aborde les structures de contrôle en PL/SQL, qui permettent de gérer le flux d'exécution des programmes. Les structures de contrôle se divisent en deux catégories principales : les **structures conditionnelles** et les **structures itératives**.

1. Structures Conditionnelles

Les structures conditionnelles permettent d'exécuter des blocs de code en fonction de certaines conditions. Elles sont essentielles pour prendre des décisions dans un programme.

- **IF THEN :**

- Exécute un bloc de code si la condition est vraie.
- Si la condition est fausse ou inconnue, le contrôle passe à l'instruction suivante après le `END IF`.
- Exemple :

```
IF condition THEN
  -- instructions
END IF;
```

- **IF THEN ELSE :**

- Permet d'exécuter un bloc de code si la condition est vraie, et un autre bloc si elle est fausse.
- Exemple :

```
IF condition THEN
  -- instructions si vrai
ELSE
  -- instructions si faux
END IF;
```

- **IF THEN ELSIF :**

- Permet de tester plusieurs conditions successivement.
- Exemple :

```
IF condition1 THEN
    -- instructions si condition1 vraie
ELSIF condition2 THEN
    -- instructions si condition2 vraie
ELSE
    -- instructions si aucune condition n'est vraie
END IF;
```

- **CASE :**

- Permet de simplifier les structures conditionnelles complexes en comparant une expression à plusieurs valeurs.
- Deux formats :
 1. **CASE expression** : Compare une expression à des valeurs spécifiques.

```
CASE expression
    WHEN valeur1 THEN
        -- instructions
    WHEN valeur2 THEN
        -- instructions
    ELSE
        -- instructions par défaut
END CASE;
```

2. **CASE WHEN condition** : Permet de tester des conditions plus complexes.

```
CASE
    WHEN condition1 THEN
        -- instructions
    WHEN condition2 THEN
        -- instructions
    ELSE
```

```
-- instructions par défaut  
END CASE;
```

2. Structures Itératives

Les structures itératives permettent de répéter un bloc de code plusieurs fois, soit un nombre défini de fois, soit jusqu'à ce qu'une condition soit remplie.

- **LOOP :**

- Boucle infinie qui nécessite une condition de sortie explicite avec `EXIT`.
- Exemple :

```
LOOP  
  -- instructions  
  EXIT WHEN condition;  
END LOOP;
```

- **WHILE :**

- Répète un bloc de code tant qu'une condition est vraie.
- Exemple :

```
WHILE condition LOOP  
  -- instructions  
END LOOP;
```

- **FOR :**

- Répète un bloc de code un nombre défini de fois, en utilisant un compteur implicite.
- Le compteur est automatiquement incrémenté à chaque itération.
- Exemple :

```
FOR compteur IN 1..10 LOOP  
  -- instructions  
END LOOP;
```

- **Remarque** : Le compteur ne peut pas être modifié manuellement dans la boucle.

Points Importants à Retenir :

1. **Évaluation Rapide (Short-circuit Evaluation)** : En PL/SQL, les conditions sont évaluées de gauche à droite, et l'évaluation s'arrête dès que le résultat est déterminé. Cela peut éviter des erreurs, comme une division par zéro dans une condition complexe.
2. **Gestion des Conditions NULL** : Les conditions NULL peuvent entraîner des comportements inattendus. Par exemple, une variable NULL dans une condition `IF` sera traitée comme fausse, ce qui peut conduire à des erreurs logiques.
3. **Boucles Infinies** : Les boucles `LOOP` doivent toujours avoir une condition de sortie (`EXIT`) pour éviter les boucles infinies.
4. **Compteur Implicite dans FOR** : Le compteur dans une boucle `FOR` est implicite et ne peut pas être modifié manuellement. Toute tentative de modification entraînera une erreur.
5. **Utilisation de CASE** : La structure `CASE` est très utile pour simplifier les conditions complexes et rendre le code plus lisible.

Conclusion

Les structures de contrôle en PL/SQL sont fondamentales pour créer des programmes dynamiques et réactifs. Elles permettent de gérer les décisions et les répétitions de manière efficace. Une bonne compréhension de ces structures est essentielle pour écrire du code PL/SQL robuste et maintenable.